

Challenge:	Order of the Phoenix
Category:	Forensics
Difficulty:	Hard
Tools Used:	Wireshark, Aperisolve

Summary

The challenge presents a network forensics scenario where a .pcap file appears to contain normal mixed traffic at first glance, including HTTP and UDP communications. However, the narrative hints that the intercepted message was deliberately fragmented across multiple transmissions, meaning no single packet or stream contains the full data. The real complexity lies not in decoding one format, but in identifying and extracting meaningful pieces from different protocol layers while avoiding intentional traps.

In brief, the procedure involves systematically analyzing HTTP traffic to extract files and responses, recognizing hidden data within an image using least significant bit steganography, interpreting a melody file as a pattern-based substitution to recover both a keyword and a numeric key, and analyzing UDP packet lengths as an encoding mechanism resembling Morse code. The final step requires applying the recovered key to decrypt raw byte data using XOR, after viewing it in hex format. By combining all recovered fragments in the correct order, the complete hidden flag is reconstructed.

Problem Statement

An encrypted transmission intercepted from Dawlish’s private infrastructure was accompanied by an internal note—one that reveals more than intended.

In it, he boasts about optimizing his encryption to improve performance, specifically by reducing the computational burden of decryption.

But in cryptography, efficiency often comes at a cost.

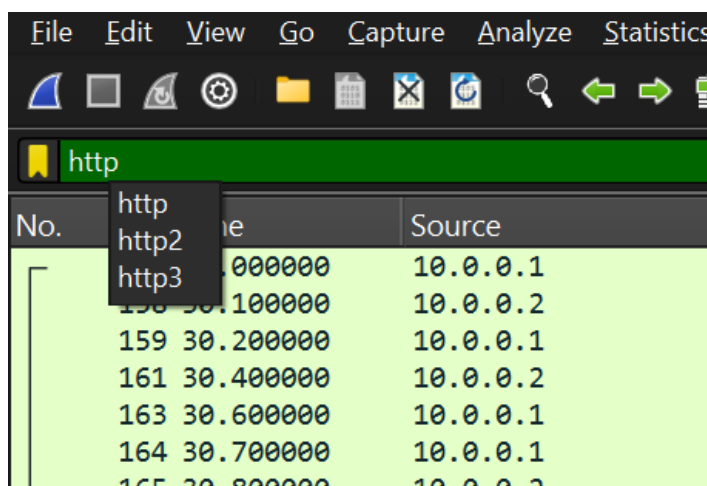
The public key and ciphertext appear standard. The flaw lies not in what is visible—but in what was altered behind the scenes.

If his “optimization” compromised the mathematical foundation, the system may be more fragile than it seems.

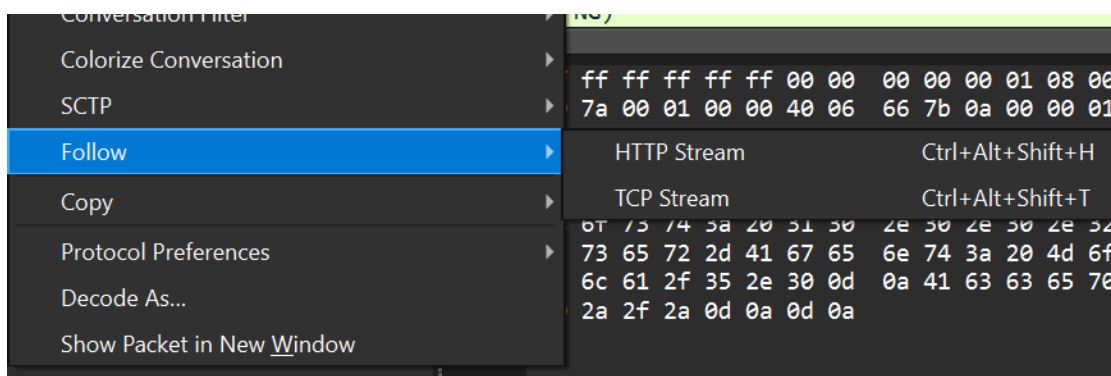
Recover the weakness. Exploit it. And reveal what was meant to stay hidden.

Steps to Solve

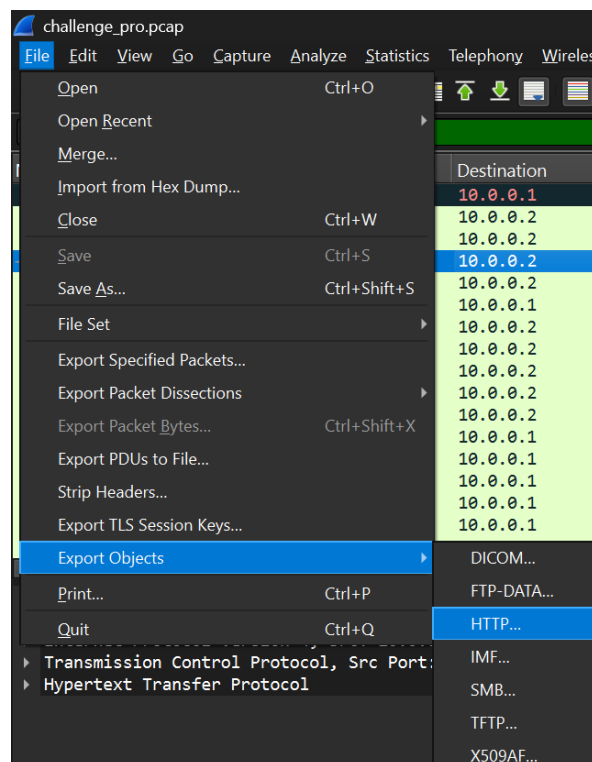
1. Open Wireshark → File → Open → challenge_pro.pcap.
2. Observe mixed traffic → data is split across multiple sources.
3. Apply filter: http



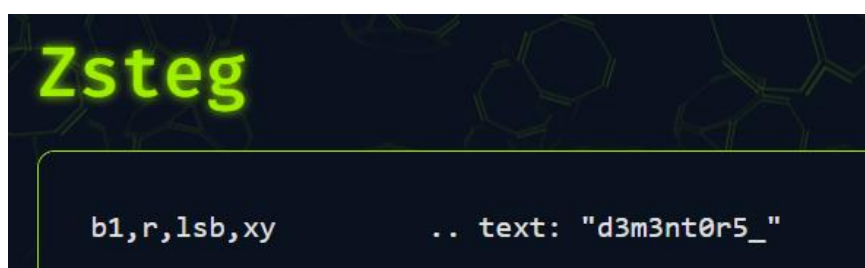
4. Identify endpoints: /api/scroll, /api/logs, /api/animation, /api/melody, /api/archive.
5. Right-click /api/scroll → Follow → TCP Stream → see CTF{ (flag start).



6. Open /api/logs → follow stream → see full flag-like string → ignore (trap, contradicts Hint 4).
7. Go to File → Export Objects → HTTP.



8. Extract frame.png → relates to Hint 1 (portrait + least significant detail).
9. Upload image to AperiSolve.
10. Check LSB/steg results → get : d3m3nt0r5_



11. Save as Fragment 1.

12. From HTTP objects, extract spell.imy.

13. Open it → observe:

BPM: 66

MELODY: g5 e g d5 g5 d5 a5 e b5 g6

Note	Position	Letter
g5	1,5	l, l ✓
e	2,8	e, e ✓
g	3	g
d5	4,6	i, i ✓
a5	7	m
b5	9	n
g6	10	s

14. Apply Hint 2 (melody contains key + word).

15. Map repeating notes → decode: legilimens

16. Save as Fragment 2 and note key: 66

17. Apply filter: udp.port == 5001

18. Observe unreadable data → apply Hint 3 (read its weight).

19. Add packet length column → right-click Frame length → Apply as Column.

20. Notice only values: 50 and 62

21. Map using Morse code:

- 50 = .
- 62 = -

22. Convert sequence → group into valid Morse → decode: 4l0h0m

....-	→ 4
.-..	→ l
-----	→ 0
....	→ h
-----	→ 0
--	→ m

23. Save as Fragment 3.

24. Open /api/archive → Follow TCP Stream → Hex Dump.

Show as Hex Dump ▼

25. Get bytes: 72 30 76 3F

26. Use key from melody: 66 → 0x42

```
72 ^ 42 → 30 → '0'
30 ^ 42 → 72 → 'r'
76 ^ 42 → 34 → '4'
3F ^ 42 → 7D → '}'
```

27. XOR each byte → result: 0r4}

28. Save as Fragment 4.

29. Combine all using Hint 4 order:

Cruxhunt{ + d3m3nt0r5_ + legilimens + _4l0h0m + 0r4}

30. Final flag:

Cruxhunt{d3m3nt0r5_legilimens_4l0h0m0r4}